Linked Lists

A linked list is a data structure where each object is stored in a *node*

As well as storing data, the node must also contains a reference/pointer to the node containing the next item of data

Linked Lists

We must dynamically create the nodes in a linked list

Thus, because `new` returns a pointer, the logical manner in which to track a linked lists is through a pointer

A `Node` class must store the data and a reference to the next node (also a pointer)

Linked Lists

The node must store data and a pointer:

```
class Node {
    private:
        int element;
        Node *next_node;
    public:
        Node( int = 0, Node * = 0 );

        int retrieve() const;
        Node *next() const;
};
```

Linked Lists

The constructor assigns the two member variables based on the arguments

```
Node::Node( int e, Node *n ):
element( e ),
next_node( n ) {
    // empty constructor
}
```

The default values for both arguments are both `0`

Linked Lists

The two member functions are accessors which simply return the `element` and the `next_node` member variables, respectively

```
int Node::retrieve() const {
    return element;
}

Node *Node::next() const {
    return next_node;
}
```

Linked Lists

Because each node in a linked lists refers to the next, the linked list class need only link to the first node in the list

The linked list class will require one member variable: a pointer to a node

```
class List {
    private:
        Node *list_head;
    // ...
};
```

Linked Lists

To begin, let us look at the internal representation of a linked list

Suppose we want a linked list to store the values

42 95 70 81

in this order

Linked Lists

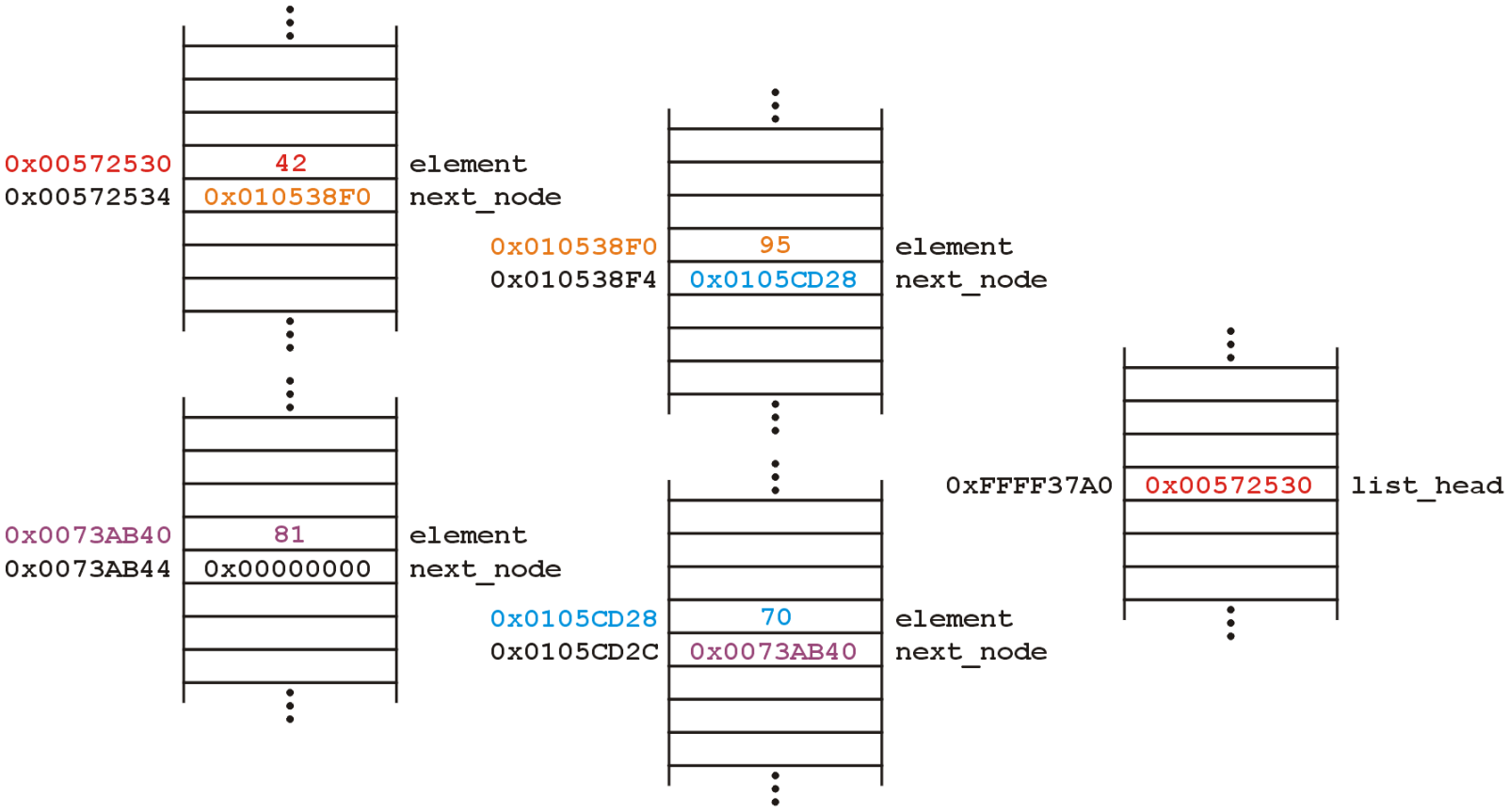
A linked list is node-based, and therefore each node may appear anywhere in memory

Also the memory required for each node equals the memory required by the member variables

- 4 bytes for the linked list (a pointer)
- 8 bytes for each node (an **`int`** and a pointer)

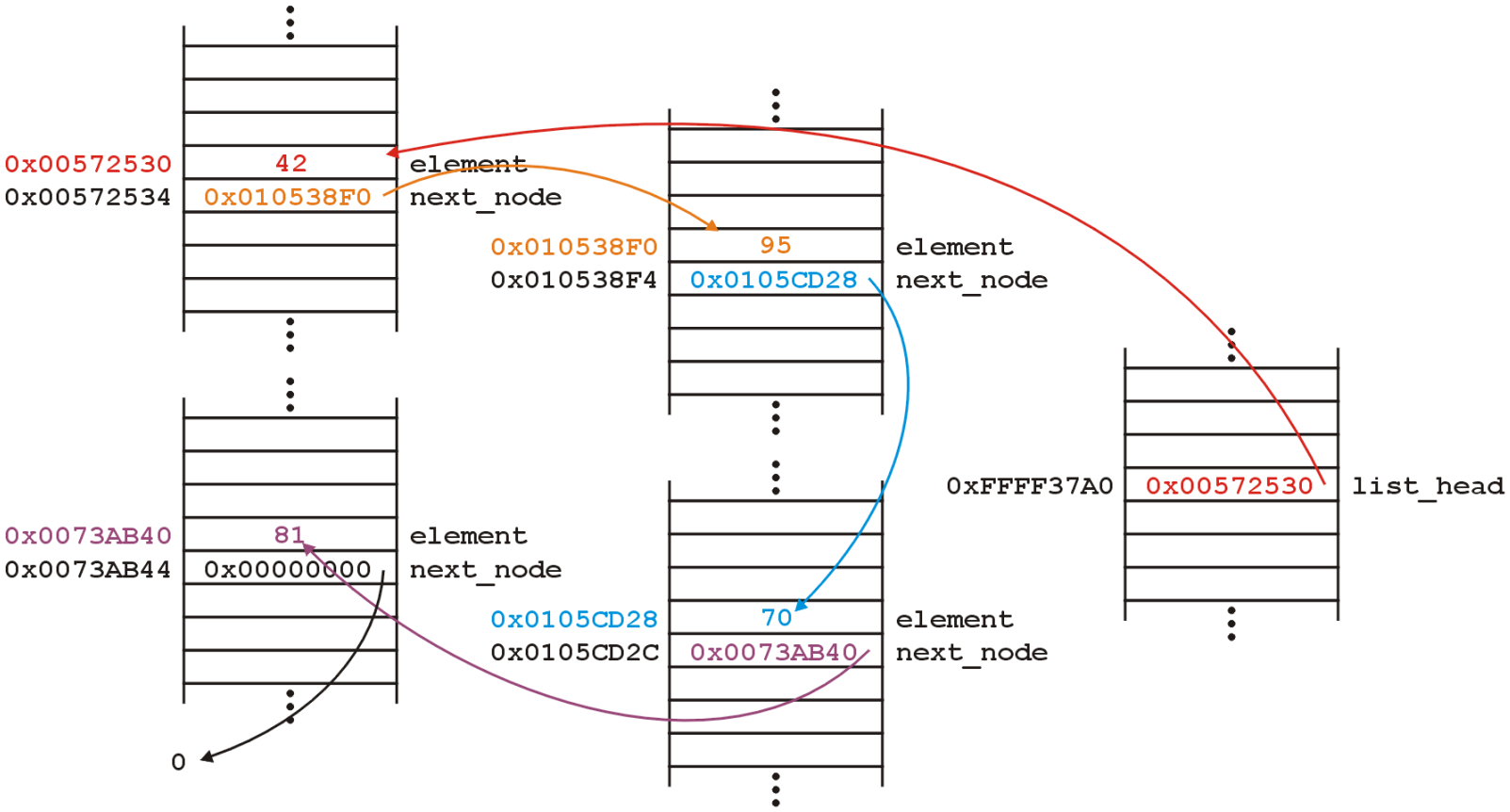
Linked Lists

Such a list could occupy memory as follows:



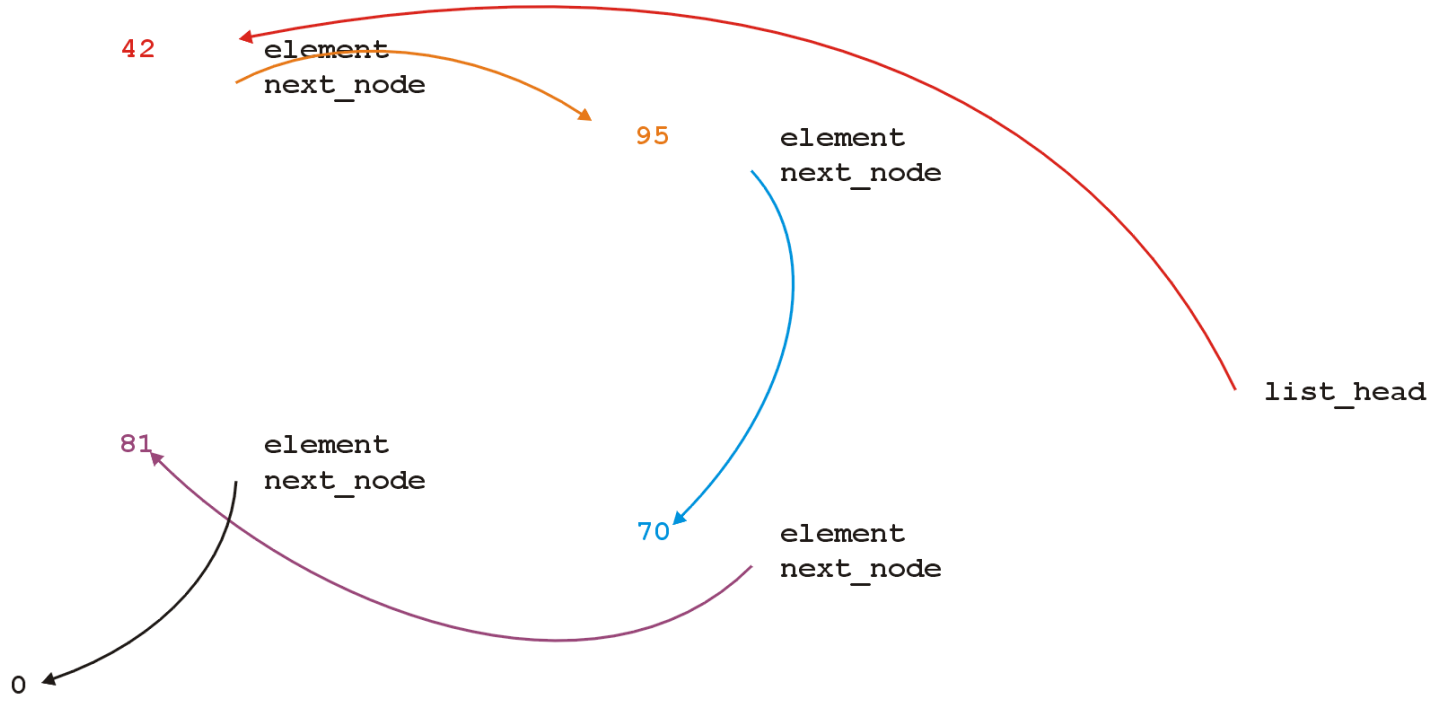
Linked Lists

Notice how each next_node address points to the next node in the list?



Linked Lists

Because the addresses are arbitrary, we can remove that information:



Linked Lists

We will clean up the representation as follows:

We do not specify the addresses because they are arbitrary and:

- the contents of the circle is the element
- the `next_node` pointer is represented by an arrow

Linked Lists

First, we want to create a linked list

We also want to be able to:

- add to,
- access, and
- delete from

the elements stored in the linked list

Linked Lists

We can do them with the following operations:

- adding, retrieving, or removing the value at the front of the linked list

```
void push_front();
int front() const;
void pop_front();
```

- we may also want to get the head of the linked list

```
Node *head() const;
```

Linked Lists

All these operations affect/access the first node of the linked list

We may want to perform operations on an arbitrary node of the linked list, for example:

- check for membership of an int in the list:

```
bool member( int ) const;
```

- remove an int from the list:

```
bool remove( int );
```

Linked Lists

Additionally, we may wish to check the state: is the linked list empty?

```
bool empty() const;
```

We will determine that the list is empty by checking if the **`list_head`** pointer is **`0`**

Linked Lists

Suppose we have a very simple linked list:

```
class List {
    private:
        Node *list_head;
    public:
        List();

        // Accessors
        bool empty() const;
        int front() const;
        Node *head() const;
        bool member( int ) const;

        // Mutators
        void push_front( int );
        int pop_front();
        bool remove( int );
};
```

Linked Lists

The constructor initializes the linked list

We do not count how may objects are in this list, thus:

- we must rely on the last pointer in the linked list to point to a special value
- in C++, that standard value is `0`

Linked Lists

Thus, in the constructor, we assign `0` to **`list_head`**

```
List::List():list_head( 0 ) {
    // empty constructor
}
```

We will always ensure that when a linked list is empty, the list head is assigned `0`

Linked Lists

A linked list may be defined in two ways:

- statically: the statement **`List ls;`** defines **`ls`** to be a linked list and the compiler deals with memory allocation
- dynamically: the statement

```
List *pls = new List();
```

requests sufficient memory from the OS to store an instance of the class

In both cases, the memory is allocated after which the constructor is called

Linked Lists

Starting with the easier member functions:

```
bool List::empty() const {
    if ( list_head == 0 ) {
        return true;
    } else {
        return false;
    }
}
```

or better yet:

```
bool List::empty() const {
    return ( list_head == 0 );
}
```

Linked List

The member function `Node *head() const` is easy enough to implement:

```
Node *List::head() const {
    return list_head;
}
```

This will always work: if the list is empty, it will return **`0`**

Linked List

To get the first element in the linked list, we must access the node to which the `list_head` is pointing

Because we have a pointer, we must use the `->` operator to call the member function:

```
int List::front() const {
    return list_head->retrieve();
}
```

Linked List

The member function **`int front() const`** requires some additional consideration, however:

- what if the list is empty?

If we tried to access a member function of a pointer set to `0`, we would access restricted memory

The operating system would terminate the running program

Linked List

Instead, we can use an exception handling mechanism where we thrown an exception

We define a class

```
class underflow {
    // emtpy
};
```

and then we *throw* an instance of this class:

```
throw underflow();
```

Linked Lists

Thus, the full function is

```
int List::front() const {
    if ( empty() ) {
        throw underflow();
    }

    return list_head->retrieve();
}
```

Linked Lists

Why is `emtpy()` better than

```
int List::front() const {
    if ( list_head == 0 ) {
        throw underflow();
    }

    return list_head->retrieve();
}
```

Two benefits:

- more readable
- if the implementation changes we do nothing

Linked Lists

Next, let us add an element to the list

If it is empty, we start with:

and, if we try to add 81, we should end up with:

Linked Lists

To visualize what we must do:

- we must create a new node which:
  - stores the value `81`, and
  - is pointing to `0`
- we must then assign its address to **`list_head`**

We can do this as follows:

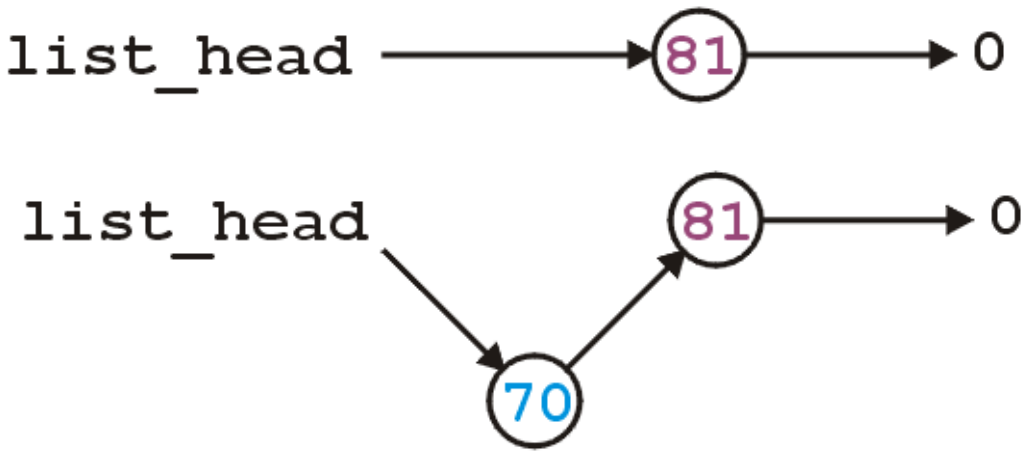
```
list_head = new Node( 81, 0 );
```

We could also use the default value...

Linked Lists

Suppose however, we already have a non-empty list

Adding `70`, we want:



Linked Lists

To achieve this, we must we must create a new node which:

- stores the value `70`, and
- is pointing to the current list head

– we must then assign its address to **`list_head`**

We can do this as follows:

```
list_head = new Node( 70, list_head );
```

Linked Lists

Thus, our implementation could be:

```
void List::push_front( int n ) {
    if ( empty() ) {
        list_head = new Node( n, 0 );
    } else {
        list_head = new Node( n, list_head );
    }
}
```

Linked Lists

We could, however, note that when the list is empty, `list_head == 0`, thus we could shorten this to:

```
void List::push_front( int n ) {
    list_head = new Node( n, list_head );
}
```

Linked Lists

Are we allowed to do this?

```
void List::push_front( int n ) {
    list_head = new Node( n, list_head );
}
```

Yes: the right-hand side of an assignment statement is evaluated first

The original value of **`list_head`** is accessed first before the function call is made

Linked Lists

Removing from the linked list is even easier:

- we simply assign the list head to the next pointer of the first node

Graphically, given:

we want:

Linked Lists

Easy enough:

```
int List::pop_front() {
    list_head = list_head->next();
}
```

Unfortunately, we have some problems:

- we want to return the item stored
- the list may be empty
- we still have the memory allocated for the node containing **70**

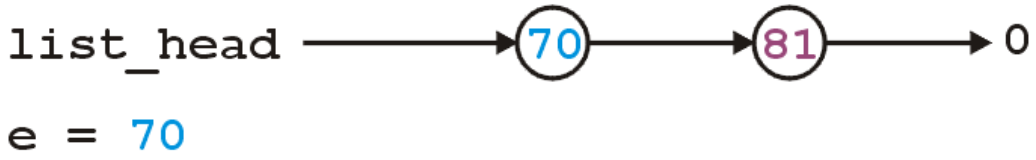
Linked Lists

```
int List::pop_front() {
    if ( empty() ) {
        throw underflow();
    }

    int e = front();
    delete list_head;
    list_head = list_head->next();
    return e;
}
```

Let's examine this...

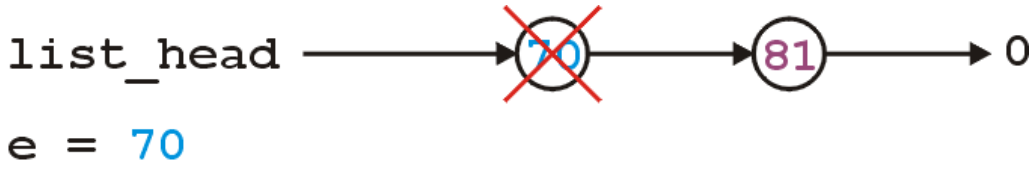
Linked Lists

```
int List::pop_front() {
      if ( empty() ) {
           throw underflow();
      }

      int e = front();

      delete list_head;

      list_head = list_head->next();

      return e;
}
```



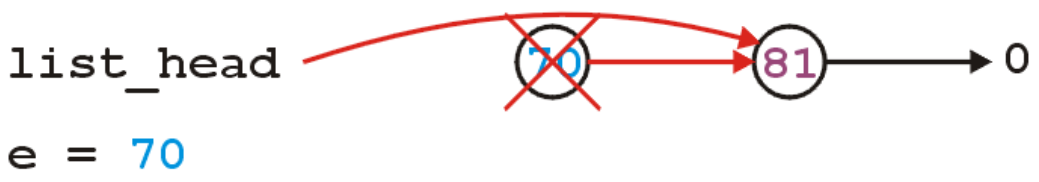
Linked Lists

```
int List::pop_front() {
      if ( empty() ) {
           throw underflow();
      }

      int e = front();

      delete list_head;

      list_head = list_head->next();

      return e;
}
```



Linked Lists

```
int List::pop_front() {
      if ( empty() ) {
           throw underflow();
      }

      int e = front();

      delete list_head;

      list_head = list_head->next();

      return e;
}
```



Linked Lists

The problem is, we are accessing a node which we have just deleted

Unfortunately, this will work more than 99% of the time:

- the running program (process) may still own the memory

Once in a while it will fail ...

... and it will be almost impossible to debug

http://xkcd.com/379/

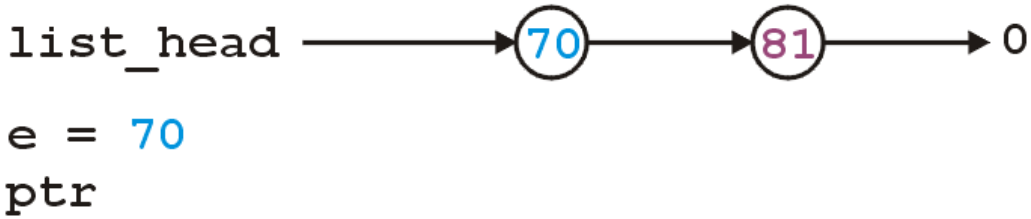
Linked Lists

The correct implementation assigns a temporary pointer to point to the node being deleted:

```
int List::pop_front() {
    if ( empty() ) {
        throw underflow();
    }

    int e = front();
    Node *ptr = list_head;
    list_head = list_head->next();
    delete ptr;
    return e;
}
```

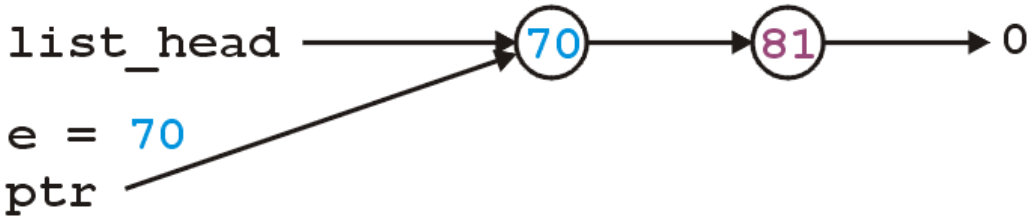
Linked Lists

```
int List::pop_front() {
    if ( empty() ) {
        throw underflow();
    }

    int e = front();

    Node *ptr = list_head;

    list_head = list_head->next();

    delete ptr;

    return e;
}
```



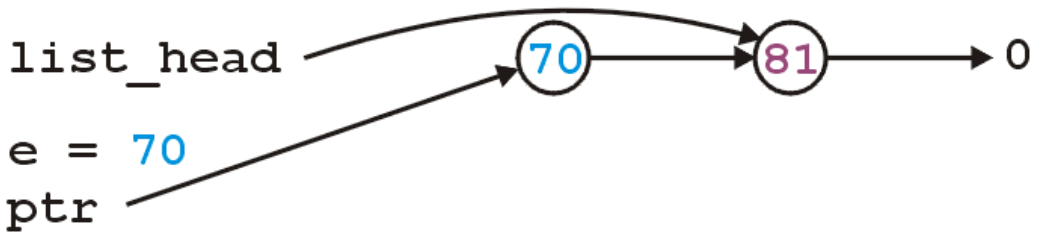
Linked Lists

```
int List::pop_front() {
    if ( empty() ) {
        throw underflow();
    }

    int e = front();

    Node *ptr = list_head;

    list_head = list_head->next();

    delete ptr;

    return e;
}
```



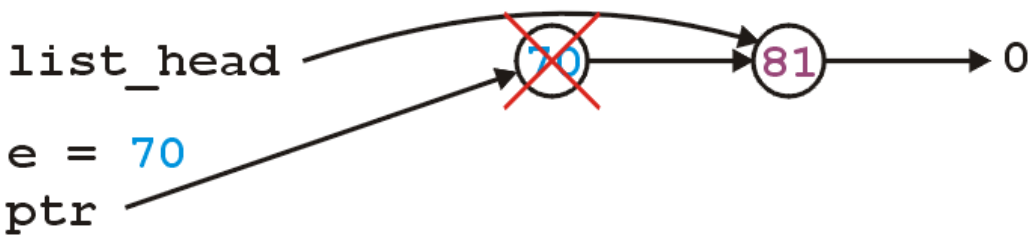
Linked Lists

```
int List::pop_front() {
    if ( empty() ) {
        throw underflow();
    }

    int e = front();

    Node *ptr = list_head;

    list_head = list_head->next();

    delete ptr;

    return e;
}
```



Linked Lists

```
int List::pop_front() {
    if ( empty() ) {
        throw underflow();
    }

    int e = front();

    Node *ptr = list_head;

    list_head = list_head->next();

    delete ptr;

    return e;
}
```



Linked Lists

The next step is to look at member functions which potentially require us to step through the entire list:

```
int count( int ) const;
int erase( int );
```

In both cases, we will return the number of nodes found

- In Project 1, you are guaranteed the objects placed into the list are unique

In the second, we will also delete the node

Linked Lists

The process of stepping through a linked list can be thought of as being analogous to a for-loop:

- we initialize a temporary pointer with the list head
- we continue iterating until the pointer equals 0
- with each step, we set the pointer to point to the next object

Linked Lists

Thus, we have:

```
for ( Node *ptr = head(); ptr != 0; ptr = ptr->next() ) {
   // do something
    // use ptr->fn() to call member functions
    // use ptr->var to assign/access member variables
}
```

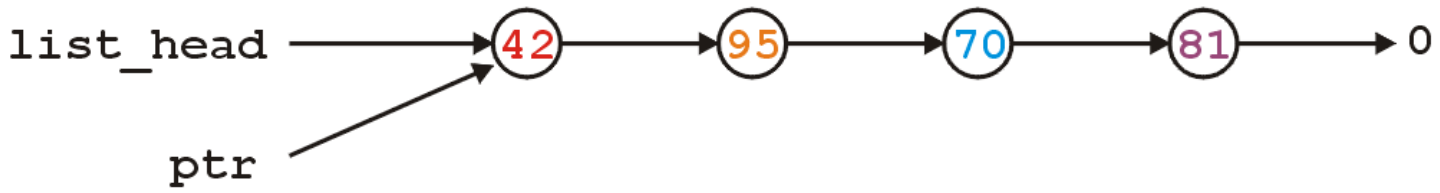
Linked Lists

Analogously:

```
for ( Node *ptr = head(); ptr != 0; ptr = ptr->next() )
for ( int i = 0;          i != N;   ++i )
```

Linked Lists

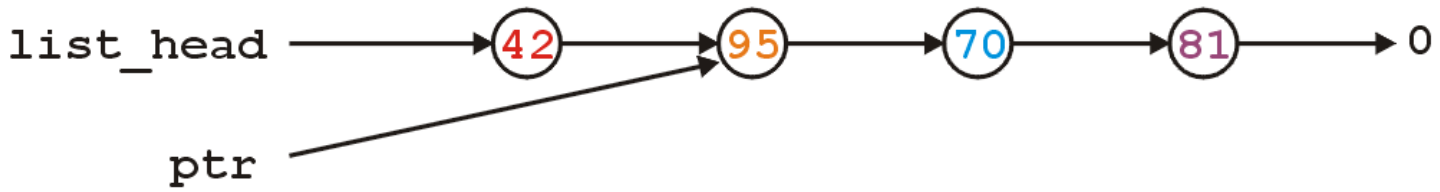
With the initialization and first iteration of the loop, we have:



`ptr != 0` and thus we evaluate the body of the loop and then set `ptr` to the next pointer of the node it is pointing to

Linked Lists

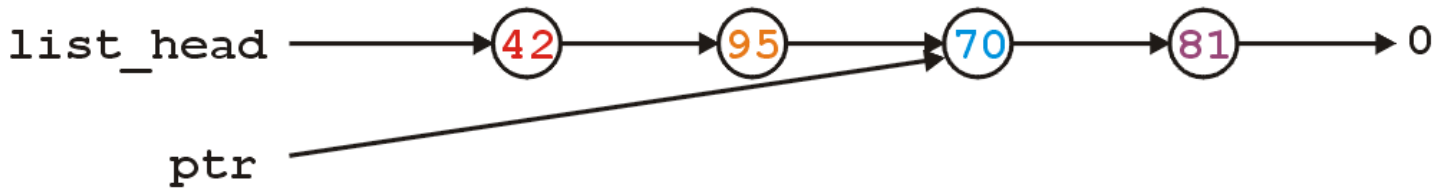
`ptr != 0` and thus we evaluate the loop and increment the pointer



In the loop, we can access the value being pointed to by using `ptr->retrieve()`

Linked Lists

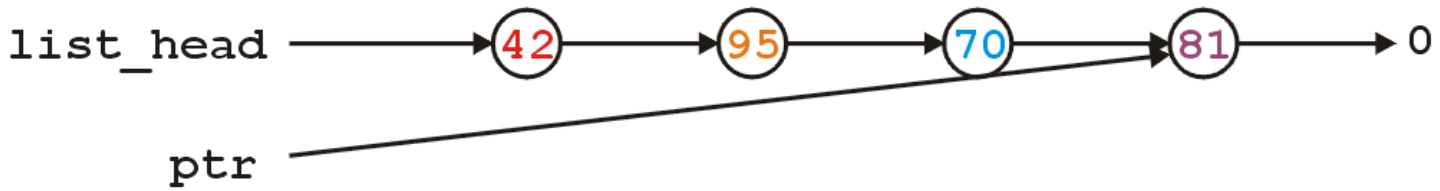
`ptr != 0` and thus we evaluate the loop and increment the pointer



Also, in the loop, we can access the next node in the list by using `ptr->next()`

Linked Lists

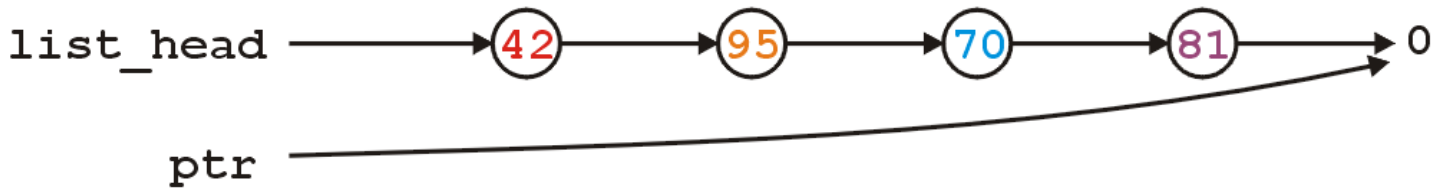
- **`ptr != 0`** and thus we evaluate the loop and increment the pointer



- This last increment causes **`ptr == 0`**

Linked Lists

- Here, we check and find **`ptr != 0`** is false, and thus we exit the loop



- Because the variable **`ptr`** was declared inside the loop, we can no longer access it

Linked Lists

To implement `int count(int) const`, we simply check if the argument matches the element with each step

If we find the argument, we return 1 (we are assuming uniqueness)

If we finish the loop, this indicates we did not find the argument: return 0

Linked Lists

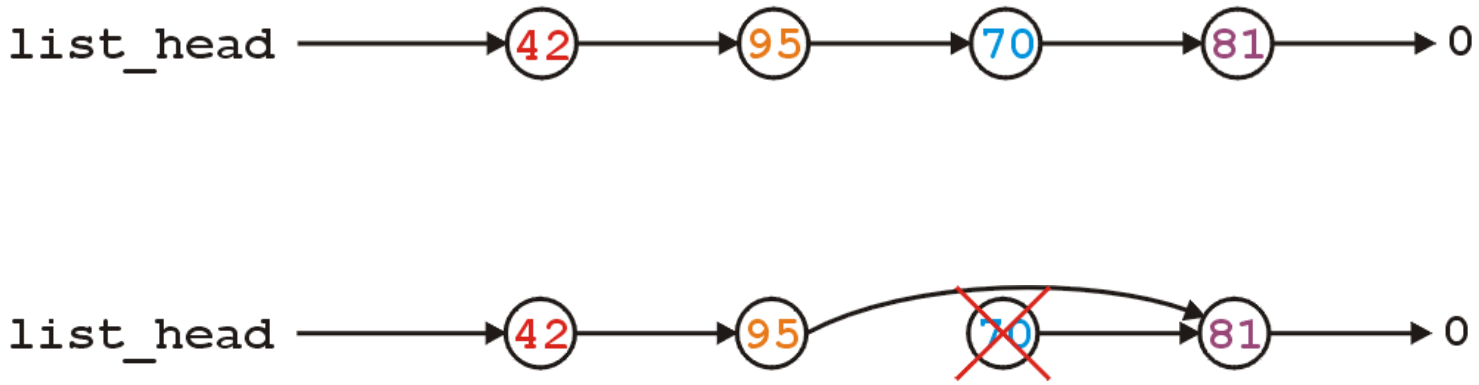
The implementation:

```
int List::count( int n ) const {
    for ( Node *ptr = list(); ptr != 0; ptr = ptr->next() ) {
        if ( ptr->retrieve() == n ) {
            return 1;
        }
    }

    return 0;
}
```

Linked Lists

- To remove an arbitrary element, i.e., to implement **`bool erase( int )`**, we must update the previous node
- For example, given

  if we delete `70`, we want to end up with



Linked Lists

- Notice that the **`remove`** function must modify the member variables of the node prior to the node being removed
- Thus, it must have access to the private instance variable **`next_node`**
- We could supply the member function

```
void setNext( Node * );
```

  however, this would be globally accessible

Linked Lists

- In Java/C#, you could define the Node class to be an *inner* class
  - this is an elegant OO solution
- In C++, you explicitly break encapsulation by declaring the class List to be a *friend* of the class Node:

```
class Node {
Node *next() const;
    // ... declaration ...
    friend class List;
};
```

Linked Lists

- Now, inside remove (a member function of **`List`**), you can modify all the member variables of any instance of the **`Node`** class

Linked Lists

- We dynamically allocated memory each time we added a new **`int`** into this list
- Suppose we delete a list before we remove everything from it
- This would leave the memory allocated with no reference to it

Linked Lists

- Thus, we need a destructor:

```
class List {
    private:
        Node *list_head;
    public:
        List();
        ~List();
        // ...etc...
};
```

Linked Lists

- The destructor has to delete any memory which had been allocated but has not yet been deallocated
- This is straight-forward enough:

```
while ( !empty() ) {
    pop_front();
}
```

Linked Lists

- Is this efficient?
- It runs in $\mathbf{O}(n)$ time, where n is the number of objects in the linked list
- Given that delete is approximately 100x slower than most other instructions (it does call the OS), the extra overhead is negligible...

Linked Lists

- Is this sufficient for a linked list class?
- Initially, it may appear yes, but we now have to look at how C++ copies objects during:
  - assignment, and
  - passing by value

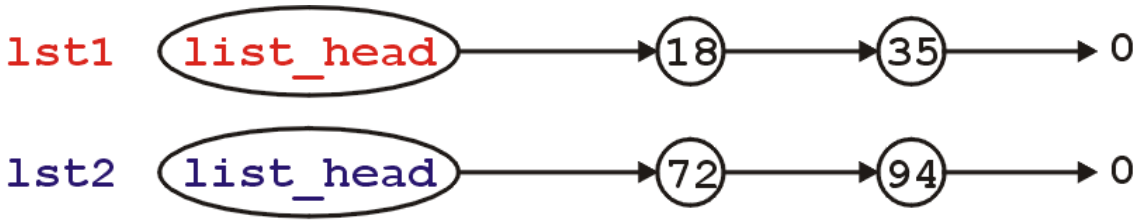
Linked Lists

- Suppose you have two objects, be they linked lists or otherwise, for example:

```
List lst1;
List lst2;
lst1.push_front( 35 );
lst1.push_front( 18 );
lst2.push_front( 94 );
lst2.push_front( 72 );
```

Linked Lists

- This is the current state:



If we assign

```
lst2 = lst1;
```

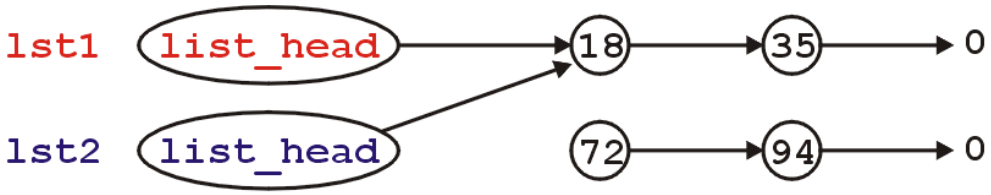
by default, all that occurs is that the member variables from `lst1` are copied to the member variables of `lst2`

Linked Lists

- Because the only member variable of this class is **`list_head`**, the value it is storing (the address of the first node) is copied over
- It is equivalent to writing:

```
lst2.list_head = lst1.list_head;
```

- Graphically:



Linked Lists

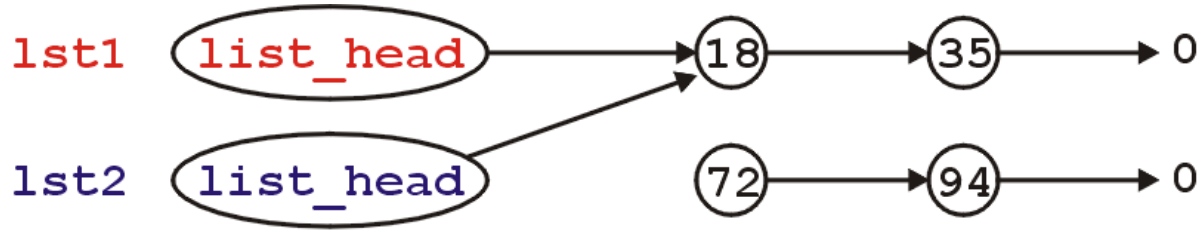
- What's wrong with this picture?
  - we no longer have links to either of the nodes storing 72 or 94
  - also, suppose we call the member function

```
lst1.pop_front();
```

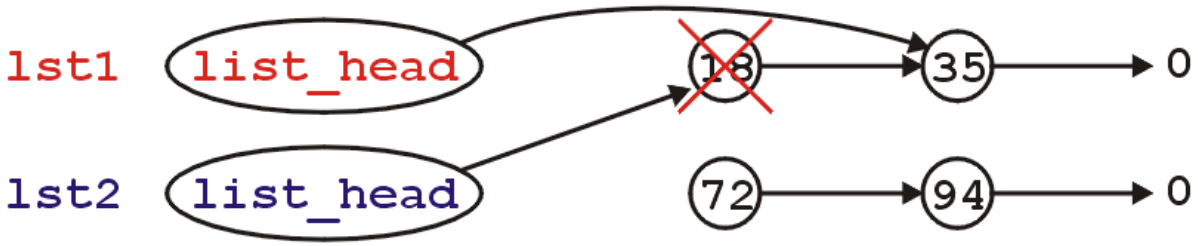
  - this only affects the member variable from the object **`lst1`**

Linked Lists

- Graphically, we go from:



to this



Linked Lists

- Now, the second list `lst2` is pointing to memory which has been deallocated...
- What is the behavior is we call

```
lst2.pop_front();
```

  ?
- The behaviour is undefined, however, soon this will probably lead to an access violation

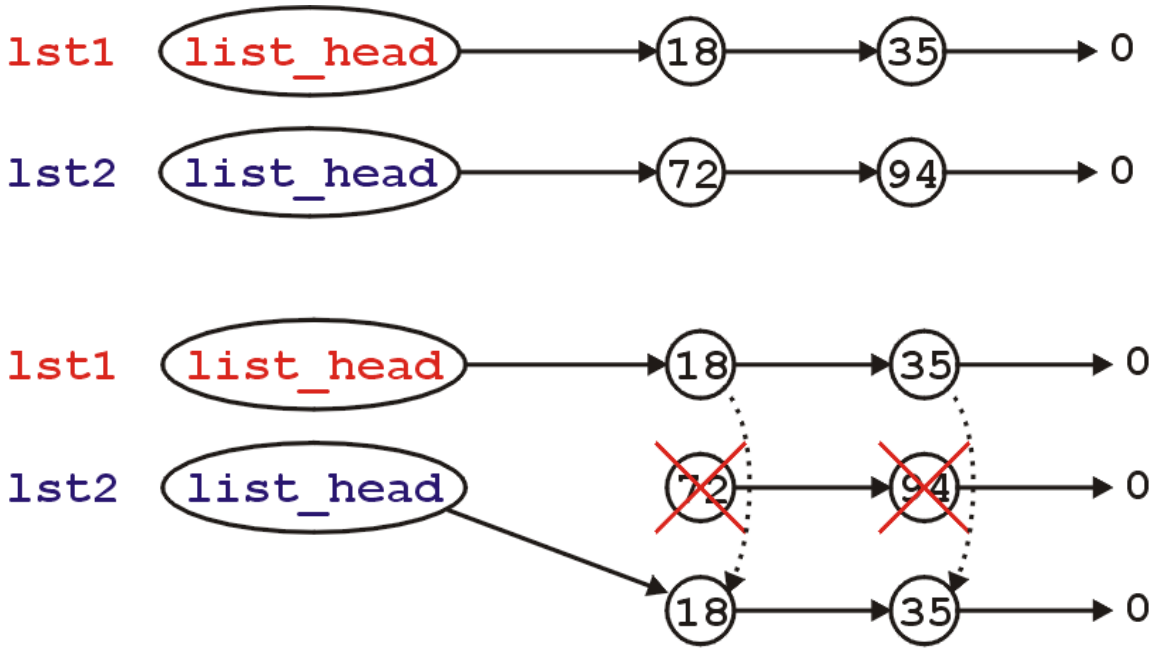
Linked Lists

- Thus, we need some way to properly implement assignment
- The logical solution would be to copy all of the nodes in the first linked list `lst1` into the second linked list `lst2`
- Unfortunately, `lst2` is not empty and therefore, we must deallocate all of its nodes first

Linked Lists

- Visually, we want to go from

to



Linked Lists

- First, to overload the assignment operator, we must overload the function named `operator =`
- This is a how you indicate to the compiler that you are overloading the `=` operator
- The signature is:

```
List & operator = ( const List & );
```

Linked Lists

- Both the passing of arguments and returning is performed by reference (the **&**)

```
List & operator = ( const List & rhs );
```

- This indicates that copies are not made (we will get to that next...)
- Because it is a reference, that means that you must refer use, for exmaple, **`rhs.head()`** or **`rhs.push_front()`**

Linked Lists

- Because we are overwriting the current object, we must:

```
List & operator = ( const List & rhs ) {
    // clean up this object
    // make a complete copy of rhs

    return *this;
}
```

- Here, the name of the argument **rhs** refers to the right-hand side of the assignment

Linked Lists

- To clean up the current object, we need remove all nodes in the linked list:

```
List & operator = ( const List & rhs ) {
    while ( !empty() ) {
        pop_front();
    }
    // make a complete copy of rhs

    return *this;
}
```

Linked Lists

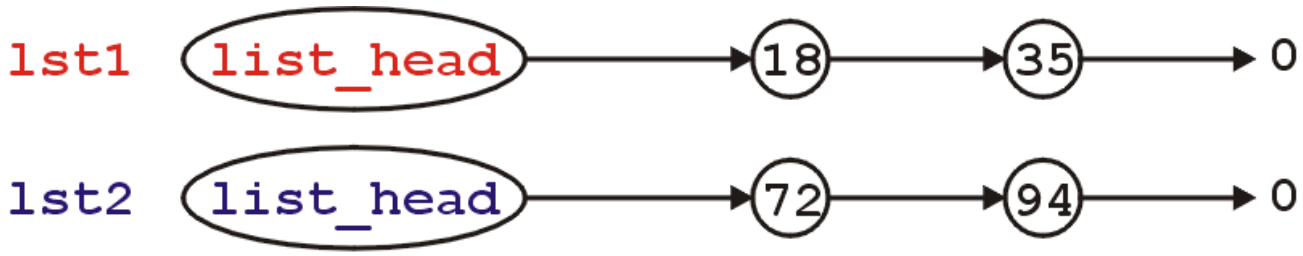
- To make a copy of the right hand side is a little more difficult...
- We must step through the second linked list and create a new node for each object in the **`rhs`** list

Linked Lists

```
List & operator = ( const List & rhs ) {
    while ( !empty() ) {
        pop_front();
    }

    if ( rhs.empty() ) {
        return *this;
    }

    list_head = new Node( rhs.front() );  // copy the front element

    for ( Node *lhptr = head(), *rhptr = rhs.head();
        rhptr->next() != 0;
        rhptr = rhptr->next(), lhptr = lhptr->next()
    ) {
        lhptr->next_node = new Node( rhptr->next()->retrieve() );
    }

    return *this;
}
```

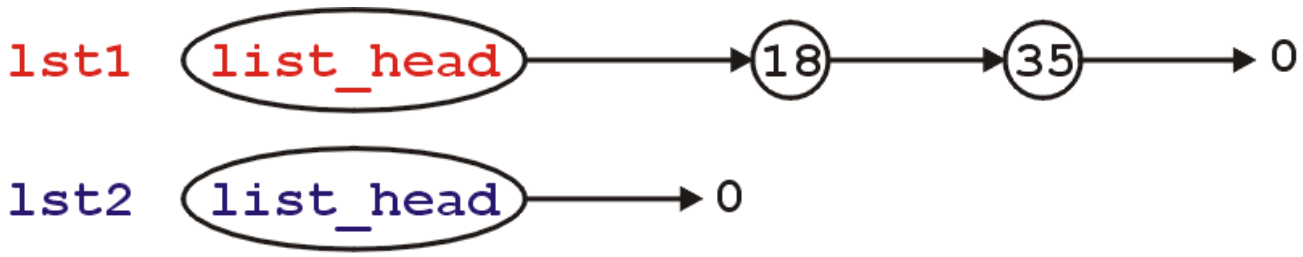
Linked Lists

- Visually, we are doing the following:
- In assigning `lst2 = lst1;`



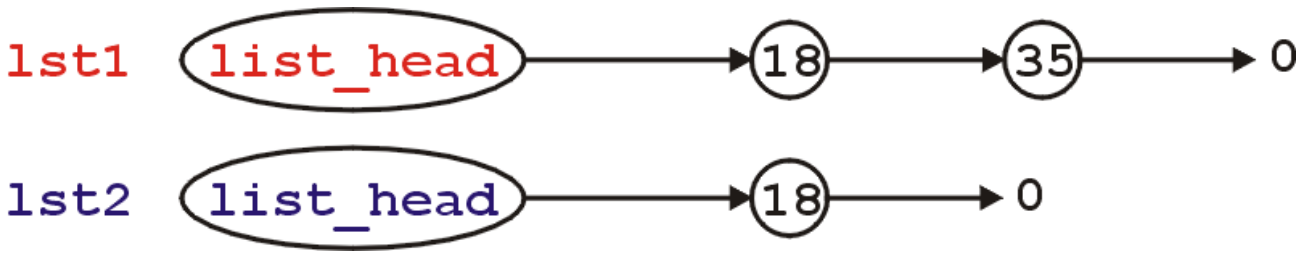
Linked Lists

- Empty the second list:



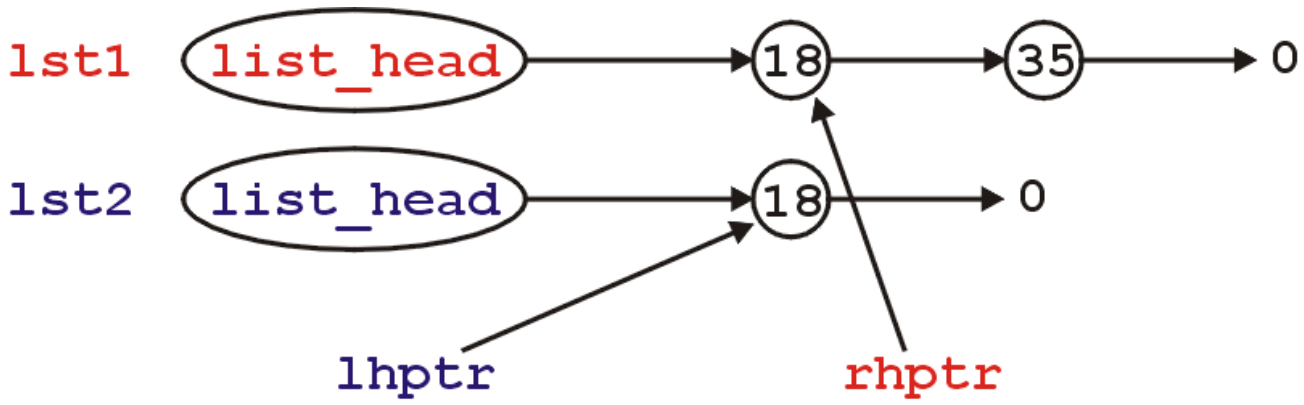
Linked Lists

- If the 1st list is not empty, push the front element of the 1st list onto the 2nd list:



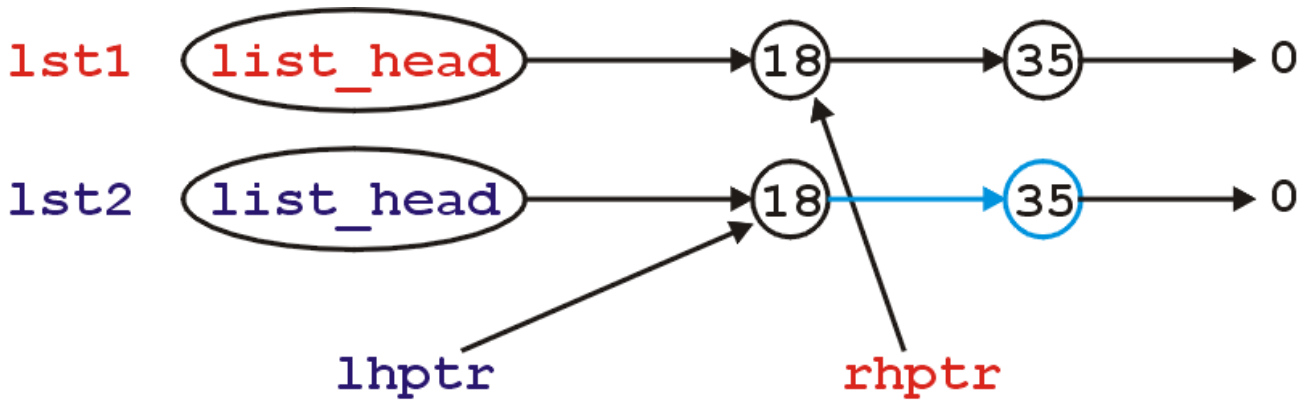
Linked Lists

- Get two pointers pointing to the first nodes in each of the lists



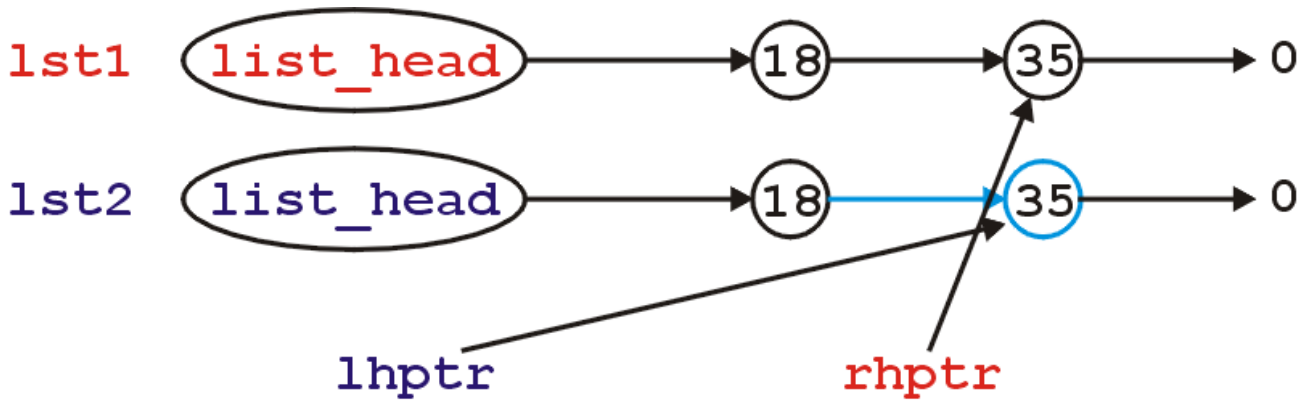
Linked Lists

- While there is a node following the node pointed to by **`rhptr`**, add that node to the 2nd list:



Linked Lists

- Then, increment each of the pointers and continue repeating this procedure until there are no more nodes to add



Linked Lists

- Why do we return *this?
- Suppose you have the assignment:

```
lst3 = lst2 = lst1;
```

- The assignment operator is right associative: it first evaluates `lst2 = lst1;`
- After this, it must assign `lst3 = ?;`
- `?` is whatever is returned by the assignment operation of `lst2 = lst1;`

Linked Lists

- With asymptotic analysis of linked lists, we can now make the following statements:

	front	arbitrary	back
insert	$\mathbf{O}(1)$	$\mathbf{O}(n)$	$\mathbf{O}(n)$ *
access	$\mathbf{O}(1)$	$\mathbf{O}(n)$	$\mathbf{O}(n)$ *
delete	$\mathbf{O}(1)$	$\mathbf{O}(n)$	$\mathbf{O}(n)$

* these become $\mathbf{O}(1)$ if we have a tail pointer